

PLAN DE PRUEBAS

StockSync — Sistema de Gestión de Inventario

Ingeniería de Software II — IUA 2026

Grupo 5: Alasino Benjamín · Calvo Tomás · Contreras Joaquín · González Martín

Versión	Fecha	Estado
1.0	03/06/2026	Baseline
1.1	03/06/2026	Actualizado — cobertura 91.37 % 48 tests PASS

1. Introducción

Este documento define el plan de pruebas para el sistema StockSync, desarrollado como trabajo de campo integrador de la materia Ingeniería de Software II (IUA, 2026). El plan sigue los lineamientos del estándar IEEE 829 y es complementario al Plan SQA del proyecto.

StockSync es un sistema de gestión de inventario orientado a comercios de indumentaria. Su backend está desarrollado en Python con FastAPI y SQLAlchemy, y su frontend en TypeScript con React. La base de datos es PostgreSQL.

1.1 Propósito

Verificar que los módulos de negocio críticos del sistema cumplen los requerimientos funcionales y no funcionales definidos en la propuesta de proyecto, con una cobertura mínima de código del 60%.

1.2 Referencias

- Propuesta de Proyecto StockSync — Grupo 5, IUA 2026
- IEEE Std 829 — Standard for Software Test Documentation
- Requerimientos: REQ-F01 al REQ-F05, REQ-NF01, REQ-NF02

2. Alcance

2.1 Qué se prueba

- Módulo de autenticación y seguridad: login, JWT, RBAC (REQ-F05)
- Módulo de productos: creación de Producto Padre, generación automática de variantes por producto cartesiano de atributos, eliminación lógica (REQ-F01)
- Módulo de stock / Kardex: cálculo de stock actual como SUM de movimientos, inmutabilidad del registro, alertas de punto de reorden (REQ-F02, REQ-F04)
- Módulo de ventas: validación de disponibilidad antes del commit, rechazo con HTTP 409 ante stock insuficiente (REQ-F03)
- Módulo de compras: creación de órdenes y recepción con generación de movimientos de ingreso (REQ-F02)
- Seguridad: hashing bcrypt, generación y decodificación de JWT (REQ-F05)

2.2 Qué no se prueba

- Interfaz de usuario (frontend React): no se incluyen pruebas E2E ni de componentes dado que las páginas principales aún están en construcción.
- Rendimiento bajo carga (REQ-NF02): se identificó como defecto DEF-002 pero su validación formal requiere una herramienta de carga (Locust, k6) y una base de datos real con datos de volumen.
- Pruebas de integración con base de datos PostgreSQL: los tests usan SQLite en memoria. La compatibilidad con PostgreSQL se verifica manualmente via Swagger UI.
- Pruebas de seguridad de penetración (OWASP): fuera del alcance de este hito.

3. Estrategia de Pruebas

3.1 Niveles de prueba

Se aplican dos niveles:

- **Pruebas unitarias de servicios:** se llaman directamente los métodos de la capa de servicio (AuthService, ProductService, StockService) contra una sesión SQLite en memoria, verificando la lógica de negocio en aislamiento.
- **Pruebas de integración de API:** se usa el TestClient de FastAPI + Starlette para realizar peticiones HTTP completas (incluyendo autenticación, serialización Pydantic y manejo de errores) contra la misma base de datos SQLite en memoria.

3.2 Técnicas aplicadas

Caja Negra — se definen entradas y se verifica la salida sin conocer la implementación:

- **Partición de equivalencia:** se agrupan entradas en clases válidas (credenciales correctas, stock suficiente) e inválidas (contraseña errónea, stock insuficiente, rol no autorizado).
- **Valores límite:** se prueban los puntos exactos de frontera, particularmente `stock == reorder_point` (alerta activa), `stock == reorder_point + 1` (sin alerta), y `quantity == 0` en venta (error de validación).

Caja Blanca — se verifica la estructura interna del código:

- **Cobertura de sentencias:** se asegura que las sentencias principales de los servicios sean ejecutadas por al menos un test (SUM del Kardex, generación de SKU, creación de movimientos).
- **Cobertura de ramas:** se prueban las bifurcaciones clave en `adjust_stock` (cantidad positiva →

`ADJUSTMENT_IN`; cantidad negativa → `ADJUSTMENT_OUT`) y en `verify_password` (correcto → `True`; incorrecto → `False`).

3.3 Herramientas

- pytest 8.3.3: framework de ejecución de tests.
- pytest-cov 6.0.0: medición de cobertura de código (reporte de líneas no cubiertas).
- httpx 0.27.2 + Starlette TestClient: cliente HTTP para pruebas de integración de la API FastAPI.
- SQLAlchemy + SQLite in-memory (StaticPool): base de datos de testing aislada, sin dependencia de PostgreSQL.

3.4 Entorno de pruebas

- Python 3.12.3 con dependencias del requirements.txt instaladas.
- Los tests se ejecutan con: `PYTHONPATH=. pytest tests/ -v --cov=app --cov-report=term-missing`
- Cada test usa su propia instancia SQLite en memoria (fixture de alcance 'function'); no hay estado compartido entre tests.
- Fixtures adicionales en `conftest.py`: `client` (TestClient con override de `get_db`), `admin_token` y `operator_token` para pruebas de API.

4. Criterios de Aceptación

4.1 Criterios de éxito

- Cobertura total de código $\geq 60\%$ sobre los módulos del paquete app/ (medida con pytest-cov). **Resultado obtenido: 91.37 %.**
- El 100 % de los casos de prueba definidos en este documento deben ejecutarse. Aquellos marcados como PASS deben pasar sin errores. **Resultado: 32/32 TC-XX PASS | 48 tests totales PASS.**
- Ningún defecto de severidad ALTA puede estar en estado Abierto al momento de la entrega.

4.2 Criterios de suspensión y reanudación

- **Suspensión:** si más del 20 % de los tests de un módulo fallan de forma continua (no flaky), se suspenden las pruebas de ese módulo y se escala a corrección.
- **Reanudación:** una vez corregidos los defectos que causaron la suspensión y verificados los casos fallidos.

4.3 Resultado de cobertura por módulo

Módulo / Archivo	Funciones cubiertas	v1.0	v1.1
app/services/auth_service.py	authenticate, create_user, get_users	73 %	95 %
app/services/product_service.py	create_product, get_product, delete_product, generate_variants, _build_sku, add_variant	59 %	93 %
app/services/stock_service.py	get_current_stock, get_stock_summary, get_low_stock_alerts, adjust_stock, create_sale, create_purchase_order, receive_purchase_order	84 %	89 %
app/core/security.py	hash_password, verify_password, create_access_token, decode_token, get_current_user, require_admin	95 %	95 %
app/api/v1/auth.py	login, get_me	100 %	100 %
app/api/v1/products.py	list_products, create_product, get_product, delete_product, generate_variants	65 %	65 %
app/api/v1/stock.py	get_stock_summary, get_low_stock_alerts, adjust_stock, create_sale, receive_purchase_order	76 %	76 %
app/schemas/*	Todos los schemas Pydantic	100 %	100 %
TOTAL	—	86.31 %	91.37 %

Nota v1.1: La mejora en auth_service (73 % $\rightarrow$ 95 %) y product_service (59 % $\rightarrow$ 93 %) se debe a la incorporación de tests de integración de API (TestClient) que ejercen los endpoints completos, incluyendo ramas de autenticación y RBAC que antes no eran alcanzadas.

5. Casos de Prueba

Los 32 casos de prueba formales (TC-01 a TC-32) se presentan a continuación. Distribución: 14 casos de Caja Negra / Partición de Equivalencia, 5 casos de Valor Límite, 13 casos de Caja Blanca. La suite completa incluye **48 tests** (32 TC-XX + 16 adicionales de servicio).

Resultado de ejecución: 48/48 PASS (pytest 8.3.3 sobre SQLite in-memory, Python 3.12.3).

ID	Req.	Técnica	Precondición	Pasos	Resultado Esperado	Res.
TC-01	REQ-F05	Caja Negra Partición equiv.	Usuario admin@test.com activo en BD.	POST /auth/login con username y password correctos.	HTTP 200. Body con access_token y token_type='bearer'.	✓ PASS
TC-02	REQ-F05	Caja Negra Partición equiv.	Usuario existe; contraseña es incorrecta.	POST /auth/login con contraseña errónea.	HTTP 401. Mensaje contiene 'incorrectos'.	✓ PASS
TC-03	REQ-F05	Caja Negra Partición equiv.	BD vacía sin usuarios.	POST /auth/login con email inexistente.	HTTP 401.	✓ PASS
TC-04	REQ-F05	Caja Negra	Ninguna.	GET /auth/me sin header Authorization.	HTTP 401.	✓ PASS
TC-05	REQ-F05	Caja Negra	admin_user creado; token JWT válido.	GET /auth/me con Bearer token válido.	HTTP 200. email='admin@test.com', role='admin'.	✓ PASS
TC-06	REQ-F05	Caja Negra Partición equiv.	Usuario admin@test.com ya existe en BD.	Llamar auth_service.create_user() con el mismo email.	HTTPException 400. Detalle: 'registrado'.	✓ PASS
TC-07	REQ-F01	Caja Negra	Sin producto con SKU 'PANT-CARGO'.	POST /products/ como Admin con datos válidos.	HTTP 201. is_active=True, base_sku correcto.	✓ PASS
TC-08	REQ-F01	Caja Negra Partición equiv.	Producto con base_sku='REM-BASE' ya existe.	POST /products/ con el mismo base_sku.	HTTP 400. Detalle menciona 'SKU'.	✓ PASS
TC-09	REQ-F05	Caja Negra RBAC	Usuario con rol operator autenticado.	POST /products/ con token de operator.	HTTP 403 Forbidden.	✓ PASS
TC-10	REQ-F01	Caja Negra	Ningún producto con ID 99999.	GET /products/99999.	HTTP 404.	✓ PASS
TC-11	REQ-F05	Caja Negra Elim. lógica	Producto activo existe.	DELETE /products/{id} como Admin. Luego GET /products/.	HTTP 204. Producto ausente del listado (no DELETE físico).	✓ PASS

ID	Req.	Técnica	Precondición	Pasos	Resultado Esperado	Res.
TC-12	REQ-F01	Caja Blanca Prod. cartesiano	Producto 'REM-BASE'. Atributos: S, M, ROJO, AZUL.	generate_variants([[S,M],[ROJO,AZUL]]).	4 variantes con SKUs únicos.	✓ PASS
TC-13	REQ-F01	Caja Blanca Cobertura de rama	Variante S-ROJO ya existe.	generate_variants([[S],[ROJO,AZUL]]).	Solo 1 variante nueva (S-AZUL). S-ROJO omitida.	✓ PASS
TC-14	REQ-F01	Caja Blanca Sentencia	Producto 'REM-BASE'; valores S y ROJO disponibles.	Llamar _build_sku('REM-BASE', [S.id, ROJO.id], db).	SKU comienza con 'REM-BASE' y contiene 'S' y 'ROJO'.	✓ PASS
TC-15	REQ-F02	Caja Blanca Sentencia (COALESCE)	Variante sin StockMovements.	get_current_stock(variant.id).	Retorna 0.	✓ PASS
TC-16	REQ-F02	Caja Blanca Sentencia (SUM)	Movimientos: +100 PURCHASE, -30 SALE, +10 ADJ_IN.	get_current_stock(variant.id).	80 (100 - 30 + 10).	✓ PASS
TC-17	REQ-F02	Caja Negra	Ninguna variante con ID 99999.	get_stock_summary(99999).	HTTPException 404.	✓ PASS
TC-18	REQ-F04	Valor límite stock == reorder_point	reorder_point=5; stock añadido=5.	get_low_stock_alerts().	variant.id aparece en alertas (below_reorder=True).	✓ PASS
TC-19	REQ-F04	Valor límite stock == reorder+1	reorder_point=5; stock añadido=6.	get_low_stock_alerts().	variant.id NO aparece en alertas.	✓ PASS
TC-20	REQ-F05	Caja Blanca Rama (qty > 0)	Variante activa; usuario admin.	adjust_stock(quantity=10, notes='...').	movement_type=ADJUSTMENT_IN, quantity=10.	✓ PASS
TC-21	REQ-F05	Caja Blanca Rama (qty < 0)	Variante activa; usuario admin.	adjust_stock(quantity=-5, notes='...').	movement_type=ADJUSTMENT_OUT, quantity=-5.	✓ PASS
TC-22	REQ-F03 REQ-F02	Caja Negra Partición equiv.	Stock=20 en Kardex.	POST /stock/sales con quantity=5.	HTTP 201. Stock post-venta=15.	✓ PASS

ID	Req.	Técnica	Precondición	Pasos	Resultado Esperado	Res.
TC-23	REQ-F03	Caja Negra Partición equiv.	Stock=0 (sin movimientos).	POST /stock/sales con quantity=1.	HTTP 409. Detalle: 'Stock insuficiente'.	✓ PASS
TC-24	REQ-F03	Valor límite quantity = 0	Variante activa.	POST /stock/sales con quantity=0.	HTTP 422 (validación Pydantic: quantity > 0).	✓ PASS
TC-25	REQ-F02 2 REQ-F03 3	Caja Blanca Sentencia Kardex	Stock=50.	create_sale() directamente con quantity=10.	StockMovement tipo SALE con quantity=-10.	✓ PASS
TC-26	REQ-F02	Caja Negra Estado inválido	Orden creada y ya recibida.	receive_purchase_order() por segunda vez.	HTTPException 400.	✓ PASS
TC-27	REQ-F02	Caja Blanca Sentencia Kardex	Orden PENDING con 50 unidades.	receive_purchase_order().	StockMovement PURCHASE quantity=50. Stock final=50.	✓ PASS
TC-28	REQ-F05	Caja Blanca Sentencia bcrypt	Ninguna.	hash_password('micontrasena123').	Resultado ≠ texto plano; comienza con '\$2b\$'.	✓ PASS
TC-29	REQ-F05	Caja Blanca Rama (correcto)	Hash generado del mismo texto.	verify_password(plain, hashed).	True.	✓ PASS
TC-30	REQ-F05	Caja Blanca Rama (incorrecto)	Hash generado con 'correcta'.	verify_password('incorrecta', hashed).	False.	✓ PASS
TC-31	REQ-F05	Caja Blanca Sentencia JWT	SECRET_KEY configurado.	create_access_token(subject=42). . Decodificar.	payload['sub']==42. 'exp' presente.	✓ PASS
TC-32	REQ-F05	Caja Blanca Rama (JWTError)	Ninguna.	decode_token('token.invalido').	HTTPException status_code=401.	✓ PASS

6. Reporte de Defectos

A continuación se documentan los defectos detectados durante el análisis del código y la ejecución de las pruebas. La severidad sigue el esquema: **ALTA** (bloquea funcionalidad crítica), **MEDIA** (impacto en calidad o funcionalidad secundaria), **BAJA** (mejora o warning).

ID	Severidad	Descripción	Pasos para reproducir	Estado
DEF-001	BAJA	<code>datetime.utcnow()</code> está deprecado en Python 3.12. Se usa en <code>security.py</code> , <code>stock_service.py</code> y en los modelos.	1. Ejecutar la app con Python 3.12. 2. Observar <code>DeprecationWarning</code> en los logs del servidor.	Abierto
DEF-002	MEDIA	<code>get_low_stock_alerts()</code> realiza N+1 queries: 1 para obtener todas las variantes activas más 1 por variante para calcular stock. Con catálogos grandes viola REQ-NF02 (< 300 ms p95).	1. Crear 200+ variantes activas. 2. GET <code>/stock/alerts/low-stock</code> con SQL logging activo. 3. Verificar que se emiten N+1 queries.	Abierto
DEF-003	MEDIA	El estado <code>CONFIRMED</code> está definido en el enum <code>PurchaseOrderStatus</code> pero ningún endpoint transiciona una orden a ese estado. Es código muerto que aparece en la documentación Swagger, generando confusión.	1. Revisar el enum <code>PurchaseOrderStatus</code> . 2. Buscar en toda la codebase asignaciones a <code>CONFIRMED</code> . 3. Confirmar que no existe ningún endpoint que use ese estado.	Abierto
DEF-004	MEDIA	<code>_build_sku()</code> construye el SKU consultando solo los IDs que existen en la BD. Si se pasan IDs inexistentes, el SKU generado es incompleto ('BASE-') sin lanzar error ni validación.	1. Llamar <code>product_service._build_sku('BASE', [9999, 8888], db)</code> . 2. Observar que retorna 'BASE-' en lugar de lanzar <code>HTTPException</code> .	Abierto
DEF-005	BAJA	<code>generate_variants()</code> omite silenciosamente las combinaciones cuyo SKU ya existe. El llamador recibe solo variantes nuevas sin saber cuántas fueron ignoradas, dificultando auditorías.	1. Crear variante REM-BASE-S-ROJO. 2. <code>generate_variants([[S],[ROJO,AZUL]])</code> . 3. Verificar que la respuesta solo tiene S-AZUL, sin indicación de omisiones.	Abierto

6.1 Resumen de defectos

Total: 5 defectos detectados. **Severidad ALTA: 0** | **MEDIA: 3** | **BAJA: 2**. Todos los defectos de severidad ALTA han sido resueltos (ninguno encontrado en esta iteración). Los defectos MEDIA y BAJA quedan registrados para el backlog del equipo.